

Building the Tangram application

for Android

Trevin Beattie

Copyright © 2026 Trevin Beattie

Tangram for Android is distributed under the terms of the Gnu Public License, version 3.0. A copy of this license is distributed along with the code.

This manual was created by Trevin Beattie and is shared according to the terms described in the Creative Commons 3.0 Attribution-ShareAlike License. The images used in the application are tangrams and as such are not subject to copyright.

Table of Contents

1	Code Organization	1
1.1	Data Classes	1
1.2	UI Classes	2
2	Creating Puzzle Assets	3
3	Building from the Source	5
3.1	Development Environment	5
3.2	Testing	5
3.3	Generating the Application Package (APK)	5
3.4	Generating Documentation	6
4	To Do	7

1 Code Organization

As with all Android projects, the `AndroidManifest.xml` file under `app/src/main/` describes the main components of the application (its activities). Some of the specifics such as the version number and supported SDK range are contained in `app/build.gradle` instead of the manifest.

All code is under the top-level package `com.xmission.trevin.android.tangram`. The sub-packages are:

- `.data` Java objects that have little or no dependencies on Android
- `.exception` Custom exceptions used for reporting invalid puzzle definitions
- `.ui` Classes that implement the user interface of the application
- `.util` Utility classes; helper functions

The puzzles that are bundled with the application are located in `app/src/main/assets/`. All other resources used by the application (UI layout, language-dependent strings, icons, etc.) are under `app/src/main/res/`.

1.1 Data Classes

Following is an overview of the high-level Java objects used in Tangram.

TangramPuzzle

This is the basic container for puzzles. It includes their name, source, list of `TangramPieces`, and methods for reading, building, and validating puzzles.

TangramPiece

An abstract class representing a single tan. It defines its shape, rotation, position, and in the case of the parallelogram whether it is flipped over. It includes a few helper methods for checking whether the edges of two tans are touching. There are subclasses of this for each type of tan.

TPoint

An x, y coordinate in $Q\sqrt{2}$ space, which is known as the Real Quadratic Field (<https://mathworld.wolfram.com/QuadraticField.html>). Coordinates are defined as $a_x + b_x\sqrt{2}$, $a_y + b_y\sqrt{2}$. Since most puzzles have tans oriented at multiples of 45° , using separate coefficients for the rational part and a multiple of $\sqrt{2}$ allows them to be integers in the vast majority of cases. This is an abstract class, with separate `ImmutableTPoint` and `MutableTPoint` implementations that are alternately used depending on whether ease of math operations or reducing Java object allocations is needed more.

- TEdge** A line segment in $Q\sqrt{2}$ space. This utility class is used to help snap tans together, test for overlaps, and is a component of **TPolygons**.
- TPolygon** A polygon in $Q\sqrt{2}$ space. This is used to find the silhouette of a puzzle and compare two puzzles to see whether they are identical (i.e. make the same shape).

1.2 UI Classes

Following are just a few of the UI components which are specialized for the Tangram application.

PieceTrayLayout

A custom layout which holds tans not yet placed on the playing field. This contains operations for dragging tans to and from the playing field, keeping track of how many pieces are available.

PlayTableView

The main play area where puzzles can be built. This contains most of the code for drawing the tans in play, moving them as the user drags within a tan boundary, rotating them as the user moves them with two fingers, moving the play field when the user drags outside of any tans, and zooming in and out of the play field when the user pinches or stretches it with two fingers.

TangramPuzzleView

A custom view that displays a static Tangram puzzle. It contains mostly the same drawing code as **PlayTableView**, but none of the interactions. This is used both to display the goal in the corner when working on a puzzle and also to show previews of puzzles when browsing the library.

2 Creating Puzzle Assets

Many puzzles can simply be created in the app. When a valid tangram is made in free-form mode, it can be saved to a folder that has been configured in the application Preferences. The program does its best to ensure the puzzle is centered and aligned so that the tans' positions use the simplest form, and for many puzzles where the tans are all oriented in multiples of 45° and lined up at their vertices the resulting puzzle definition can be used without modification. To add these puzzles to the program assets, copy the user puzzle file from your emulator or device to the `app/src/main/assets/` folder in your development environment.

Some puzzles may require manual tweaking, especially if their edges don't line up at the vertices or they are rotated at offsets of 15° or 30° from the normal 45° angles. For these cases I've used a vector drawing program with templates of the tans laid out on a grid, and each tan has grid lines and bisecting lines drawn through it to help measure out distances between them. Then measurements are taken by counting out grid points for each coefficient: orthogonal lines count for the a coefficient, and diagonals count for the b coefficient. For reference, here are the measurements for the tans in puzzle units:

Large Triangle

The long edge is 24 units wide, and the two short edges $12\sqrt{2}$ units long. Its centroid is 4 units from the bottom edge and 8 units from the right vertex, $4\sqrt{2}$ units from each short edge and $8\sqrt{2}$ units from the acute vertices.

Medium Triangle

The long edge is $12\sqrt{2}$ units wide, and the two short edges 12 units long. Its centroid is $2\sqrt{2}$ units from the bottom edge and $4\sqrt{2}$ units from the right vertex, 4 units from each short edge and 8 units from the acute vertices.

Small Triangle

The long edge is 12 units wide, and the two short edges $6\sqrt{2}$ units long. Its centroid is 2 units from the bottom edge and 4 units from the right vertex, $2\sqrt{2}$ units from each short edge and $4\sqrt{2}$ units from the acute vertices.

Square

The edges are $6\sqrt{2}$ units long. Its centroid is $3\sqrt{2}$ units from the edges and 6 units from the vertices.

Parallelogram

The long edges are 12 units long and the short edges $6\sqrt{2}$. Its centroid is 3 units from the long edges, 6 units from the short edges, $3\sqrt{2}$ units from the obtuse vertices, and $3\sqrt{10}$ units from the acute vertices.

The coordinate system is oriented with x going from left (negative) to right (positive), and y going from top (negative) to bottom (positive). Rotation is therefore defined with positive values going clockwise. Due to the use of $Q\sqrt{2}$ coefficients rotating tans by 45° involves simple addition and division while swapping the coefficients:

$$\begin{aligned}a'_x &= b_x - b_y \\b'_x &= \frac{a_x}{2} - \frac{a_y}{2} \\a'_y &= b_x + b_y \\b'_y &= \frac{a_x}{2} + \frac{a_y}{2}\end{aligned}$$

For other angles, it is recommended to start with the coefficients best lined up to the nearest 90° angle at the point of rotation and then apply the standard trigonometric transformations to the relative coordinate of the centroid:

$$\begin{aligned}x' &= x \cdot \cos(\theta) - y \cdot \sin(\theta) \\y' &= x \cdot \sin(\theta) + y \cdot \cos(\theta)\end{aligned}$$

If the resulting rotation ends up changing the width or height of the tangram such that it is not in integer $Q\sqrt{2}$ space, you may want to add a bit of padding to one side to make the majority of tans line up. Several of the puzzles bundled with this app include an extra `size_comment` field describing their alignments.

3 Building from the Source

When making any changes to the code, you also need to change the release date in the “About...” dialog (`res/values/strings.xml: InfoPopupText`) as well as the application’s internal version number (`app/build.gradle: android.defaultConfig.versionCode` and `.versionName`).

3.1 Development Environment

Building the application `.apk` has been done using:

- Java 21 (bundled with Android Studio)
- Android Studio Quail (2026.1.1), downloaded from the official Android Studio (<https://developer.android.com/studio/>) web site.
- Gradle 9.4.1, installed by running `./gradlew wrapper --gradle-version=9.4.1`.
- The Android Gradle Plugin version 9.2.1, as specified in the file `gradle/libs.versions.toml`.

See <http://developer.android.com/sdk/index.html> for information on how to set up a project using the Android SDK.

3.2 Testing

There are a variety of automated tests both stand-alone and instrumented for testing on emulators. These are in the `app/src/test` and `app/src/androidTest` directories respectively.

Running the stand-alone tests can be done from the command line:

```
./gradlew :app:testDebugUnitTest
```

For the instrumented tests, it’s recommended that you run these in Android Studio where you can manage the virtual devices that the tests run on. At the moment the only instrumented tests are those which check the puzzle asset files; these do not have any dependencies on SDK features other than reading the assets, so it should only be necessary to run them on a device with the minimum SDK level (23) and target SDK (37).

3.3 Generating the Application Package (APK)

In Android Studio, click on the Build menu then “Build APK”. If there were no errors, this should produce `app/release/app-release.apk` (in Android Studio 2024+). You should rename this file to a more descriptive name like `tangram-1.0.0.apk`.

Alternatively from the command line, run the following commands:

```
./gradlew assembleRelease
```

This will produce an *unsigned* APK in `app/build/outputs/apk/release/app-release-unsigned.apk`. To sign this, you will need to

locate the build tools for the version specified in `app/build.gradle` under `android.buildToolsVersion` (if present; otherwise the most recent build tools supported by the Android Gradle plugin). You will also need to know where the keystore of your app signing key is located.

```

${BUILD_TOOLS_VERSION_DIRECTORY}/apksigner sign \
  --alignment-preserved \
  --ks ${ANDROID_KEYSTORE} --ks-key-alias ${SIGNING_KEY_ALIAS} \
  --out app/${APP_NAME}-${VERSION}.apk \
  app/build/outputs/apk/release/app-release-unsigned.apk

```

(The `--alignment-preserved` option may be needed if you are using build tools 35 or higher.)

3.4 Generating Documentation

Lastly, if you need to generate a new edition of this manual or the user manual, you will need `texinfo` (<http://www.gnu.org/software/texinfo/>) and `texi2pdf` (https://www.gnu.org/software/texinfo/manual/texinfo/html_node/Format-with-texi2dvi-or-texi2pdf.html) and/or `texi2html` (<http://www.nongnu.org/texi2html/>). If you have `make` there is a `Makefile` which will make running `texinfo` more convenient.

To generate the manuals in PDF, simply run:

```
make pdf
```

if you have `make`, or:

```
texi2pdf Building_Tangram.texinfo
texi2pdf Tangram.texinfo
```

To generate the manuals in HTML, run:

```
make html
```

or:

```
texi2html --split node Building_Tangram.texinfo
texi2html --split node Tangram.texinfo
```

which will produce the manuals under `doc/Building_Tangram/Building_Tangram.html` and `doc/Tangram/Tangram.html`.

4 To Do

- Polish the look of the UI
- Write up a brief user guide
- Add more puzzles